Name: Naveen Kumar P **Reg. No.:** 192412326

Experiment No.: 1

Experiment Question

Implement and demonstrate the FIND-S algorithm for finding the most specific hypothesis based on a given set of training data samples.

Aim

To implement the FIND-S algorithm in Python and use it to find the most specific hypothesis that is consistent with all the positive training examples in a given dataset.

Concept

Concept learning is the task of inferring a Boolean-valued function (a target concept) from training examples. A hypothesis is a conjunction of constraints on the attributes. Each constraint can be:

- a specific value (for example, Sunny),
- ?, meaning any value is acceptable,
- 0 (or ∅), meaning no value is acceptable.

Hypotheses are ordered from **most specific** to **most general**. FIND-S starts with the most specific hypothesis and generalizes it only as far as needed to cover each positive example. It **ignores all negative examples**. If the target concept is contained in the hypothesis space and the data is noise-free, the final hypothesis is the most specific hypothesis consistent with all positive examples.

Dataset Description

The EnjoySport dataset is used. It is stored in a CSV file named `enjoysport.csv`. It has 4 training examples, 6 input attributes and 1 target attribute (EnjoySport = Yes/No).

Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
Sunny	Warm	Normal	Strong	Warm	Same	Yes
Sunny	Warm	High	Strong	Warm	Same	Yes
Rainy	Cold	High	Strong	Warm	Change	No
Sunny	Warm	High	Strong	Cool	Change	Yes

Contents of enjoysport.csv:

```
Sky,AirTemp,Humidity,Wind,Water,Forecast,EnjoySport
Sunny,Warm,Normal,Strong,Warm,Same,Yes
Sunny,Warm,High,Strong,Warm,Same,Yes
Rainy,Cold,High,Strong,Warm,Change,No
Sunny,Warm,High,Strong,Cool,Change,Yes
```

Algorithm

```
FIND-S Algorithm
1. Initialize h to the most specific hypothesis in H:
    h = <0, 0, 0, 0, 0, 0>
2. For each positive training example x:
    For each attribute constraint a_i in h:
      If a_i is satisfied by x:
        do nothing
      Else:
        replace a_i in h by the next more general constraint satisfied by x
3. Output the hypothesis h
```

Step-by-step Procedure

1. Import the csv module and read the training data from enjoysport.csv.
2. Separate the attribute values and the target value (last column) of each example.
3. Initialize the hypothesis h with '0' for every attribute (most specific hypothesis).
4. Take the training examples one by one.
5. If the example is negative (No), ignore it and keep h unchanged.
6. If the example is positive (Yes):
    - For the first positive example, replace every '0' in h with the corresponding attribute value.
    - For later positive examples, compare each attribute of h with the example. If they match, keep it. If they differ, replace it with '?'.
7. Print the hypothesis after every example.
8. After all examples are processed, print the final (most specific) hypothesis.

Formulae

For each positive example x and each attribute i, the hypothesis h is updated as:

- If $h(i) = 0$, then $h(i) = x(i)$
- If $h(i) = x(i)$, then $h(i)$ stays the same
- If $h(i) \neq x(i)$, then $h(i) = ?$

A hypothesis h covers an example x if, for every attribute i, $h(i) = ?$ or $h(i) = x(i)$.

Generalization order: 0 (most specific) → specific value → ? (most general).

Python Program

```
import csv

# Step 1: Read the training data from the CSV file
with open('enjoysport.csv') as f:
    reader = csv.reader(f)
    header = next(reader)
    data = [row for row in reader]

print("Training Data:")
print(header)
for i, row in enumerate(data, 1):
    print(i, row)

# Step 2: Initialize the most specific hypothesis
hypothesis = ['0'] * (len(header) - 1)
print("\nInitial Hypothesis:", hypothesis)

# Step 3: FIND-S algorithm
for i, row in enumerate(data, 1):
    if row[-1] == 'Yes':                # consider only positive examples
        for j in range(len(hypothesis)):
            if hypothesis[j] == '0':
                hypothesis[j] = row[j]   # first positive example
            elif hypothesis[j] != row[j]:
                hypothesis[j] = '?'      # generalize
    print(f"After instance {i} ({row[-1]}):", hypothesis)

# Step 4: Final result
print("\nMost Specific Hypothesis:", hypothesis)
```

Output

```
Training Data:
['Sky', 'AirTemp', 'Humidity', 'Wind', 'Water', 'Forecast', 'EnjoySport']
1 ['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same', 'Yes']
2 ['Sunny', 'Warm', 'High', 'Strong', 'Warm', 'Same', 'Yes']
3 ['Rainy', 'Cold', 'High', 'Strong', 'Warm', 'Change', 'No']
4 ['Sunny', 'Warm', 'High', 'Strong', 'Cool', 'Change', 'Yes']

Initial Hypothesis: ['0', '0', '0', '0', '0', '0']
After instance 1 (Yes): ['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same']
After instance 2 (Yes): ['Sunny', 'Warm', '?', 'Strong', 'Warm', 'Same']
After instance 3 (No): ['Sunny', 'Warm', '?', 'Strong', 'Warm', 'Same']
After instance 4 (Yes): ['Sunny', 'Warm', '?', 'Strong', '?', '?']

Most Specific Hypothesis: ['Sunny', 'Warm', '?', 'Strong', '?', '?']
```

Result / Conclusion

The FIND-S algorithm was implemented in Python and run on the EnjoySport dataset. The negative example was ignored, and the hypothesis was generalized only where positive examples differed. The most specific hypothesis obtained is:

⟨Sunny, Warm, ?, Strong, ?, ?⟩

This means the person enjoys the sport when the sky is Sunny, the air temperature is Warm and the wind is Strong, whatever the humidity, water temperature and forecast. Hence the FIND-S algorithm was successfully demonstrated.